Introduction to Convolutional Neural Networks

FRC-EVL

04/03/26

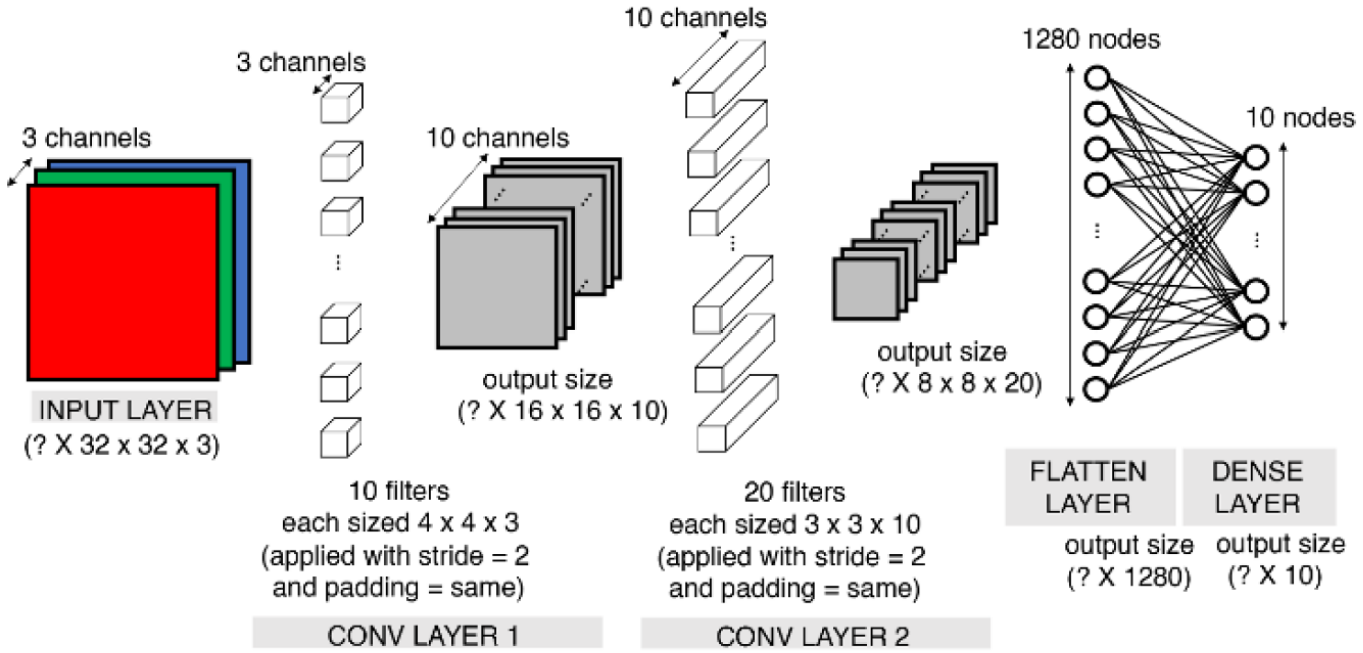
Motivation

- Computer vision has evolved rapidly with deep learning.
- Images are used as input data where pixels represent features.
- Example: a 1 megapixel image has about 3 million features.
- A fully connected network with such inputs would require billions of parameters.
- Convolutional Neural Networks (CNNs) solve this problem efficiently.

Main Components of CNNs

Three main types of layers in a CNN:

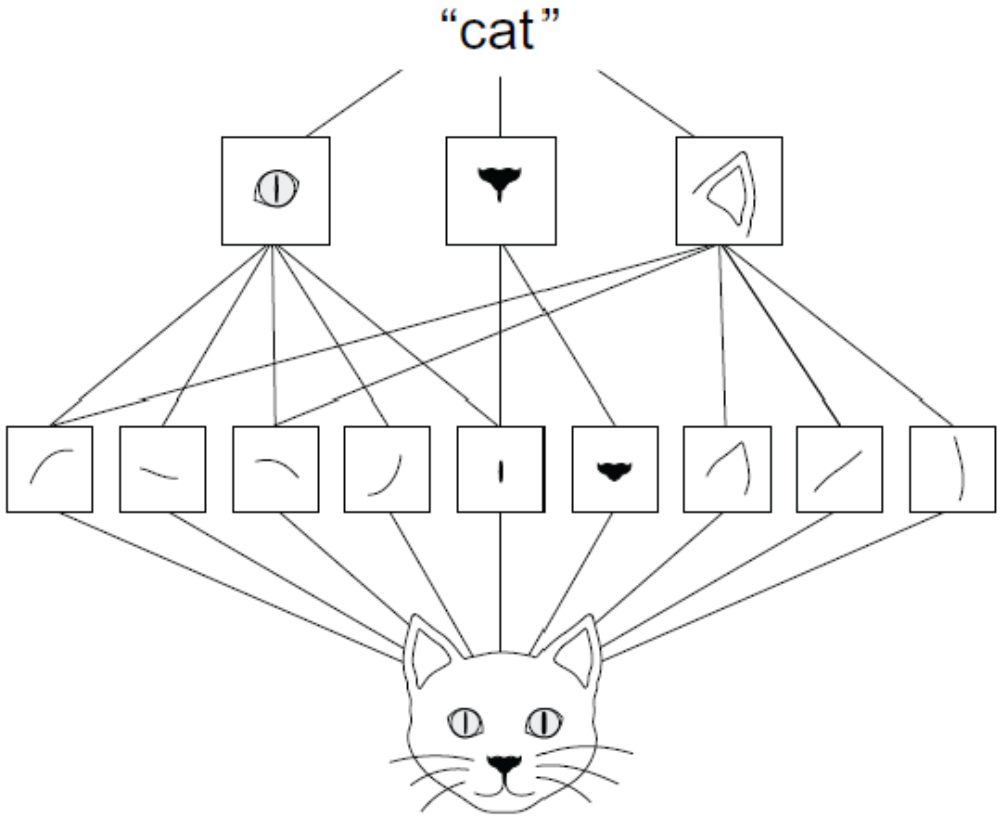
- Convolutional Layer (CONV)
- Pooling Layer (POOL)
- Fully Connected Layer (FC)



These layers are combined to learn hierarchical representations of images.

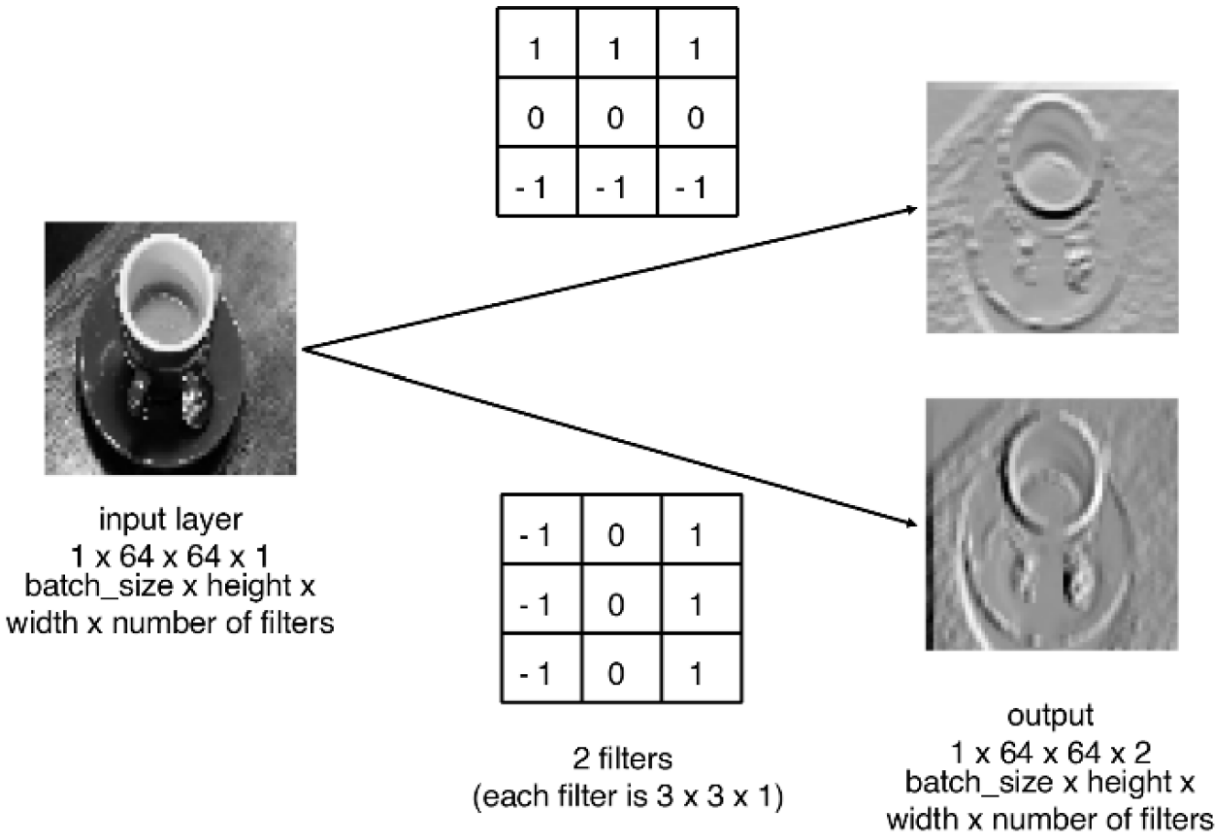
Convolutional Layer

- A convolution extracts features from an input image.
- Images are represented as matrices of pixel values.
- Filters (kernels) slide across the image.
- Each filter detects specific features such as edges or textures.

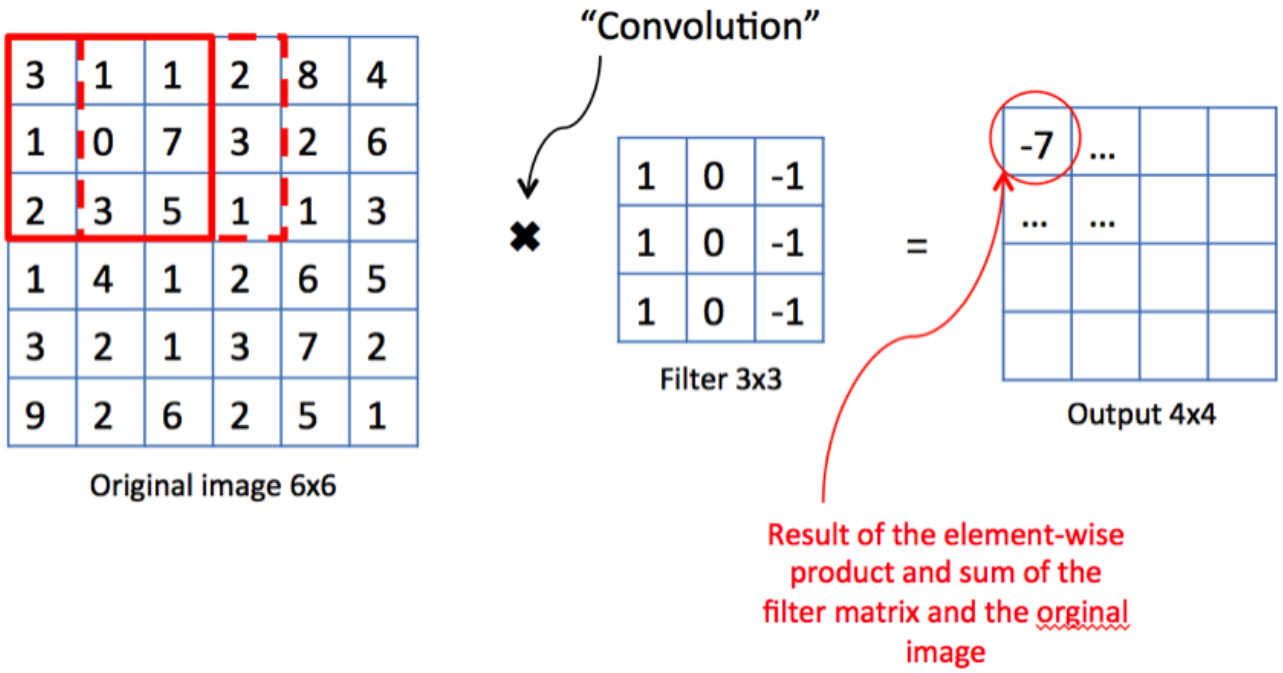


How Convolution Works

- A filter is applied to a small region of the image.
- Element-wise multiplication is performed.
- The results are summed to produce a single output value.
- Repeating this process across the image generates a feature map.



Convolution Filter



Filters as Learnable Weights

In a convolutional neural network, the filter coefficients are the **learnable parameters** of the model.

- Each filter contains a set of coefficients.
- These coefficients act as the **weights** of the network.
- During training, the network updates these weights using **backpropagation**.
- The goal is to learn filters that detect meaningful patterns such as edges, textures, or shapes.

Convolution Filters as Network Parameters

A convolution operation can be written as:

$$z_{i,j} = \sum_{u,v,c} w_{u,v,c}\, x_{i+u,j+v,c} + b$$

- x : input image
- w : filter coefficients
- b : bias

u and v index the spatial position inside the filter, while c indexes the input channels (e.g., RGB).

Key idea:

- The coefficients w are the **weights of the neural network**.
- They are learned automatically during training.

Weight Sharing in Convolutional Layers

Shared Weights

In a convolutional layer, the same filter is applied across the entire image.

- Each filter has a set of coefficients (weights).
- These weights are **shared across all spatial locations**.
- The filter detects the same feature everywhere in the image.

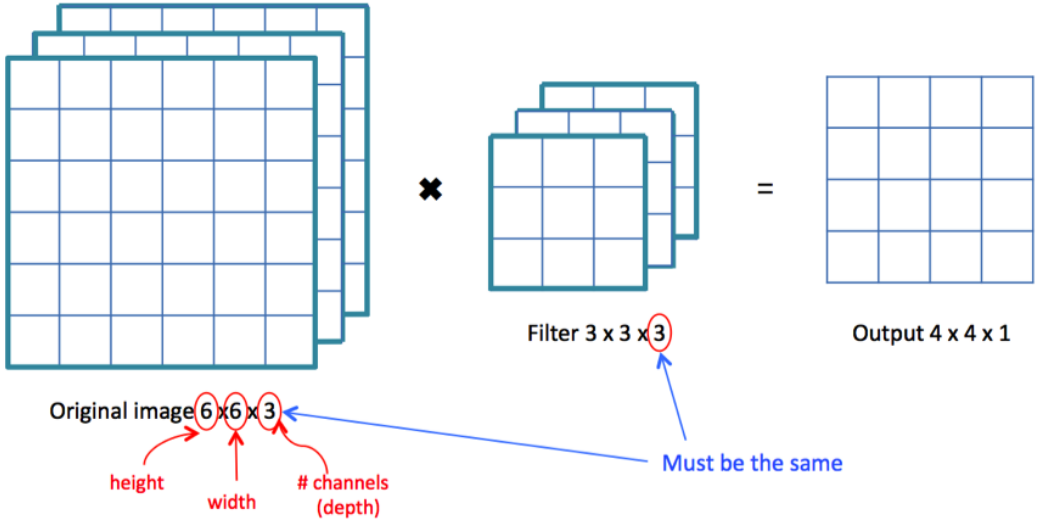
Why is this important?

- It drastically reduces the number of parameters.
- It exploits the fact that visual features (edges, textures, shapes) can appear anywhere in the image.

Filter Depth in Convolutional Layers

Key idea

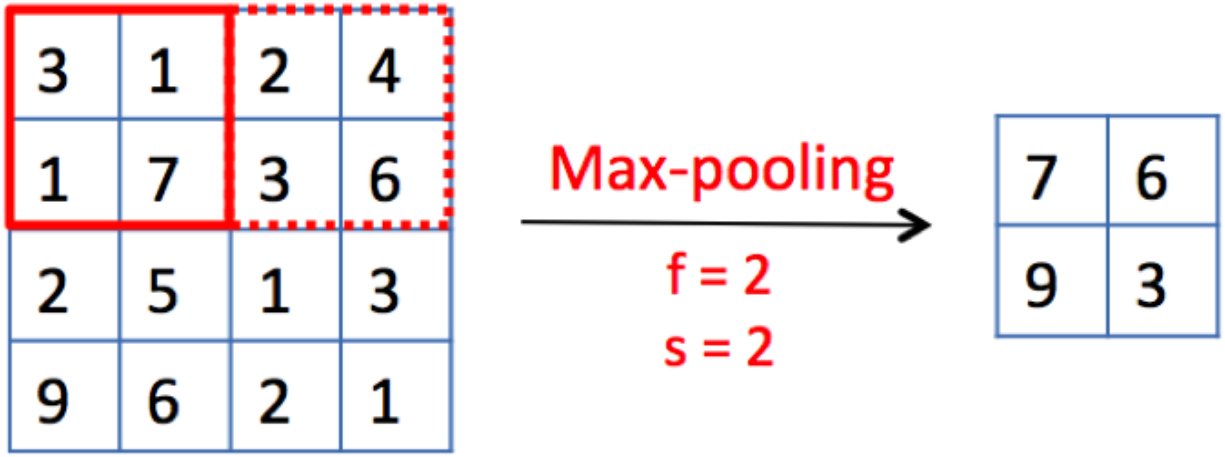
The depth of the convolutional filter must match the depth of the input activation volume.



- The filter spans **all channels of the input**.
- The convolution combines spatial information and channel information.

Stride

- Stride defines how far the filter moves each step.
- Example:
  - Stride = 1: filter moves one pixel at a time.
  - Stride = 2: output size becomes smaller.
- Increasing stride reduces the spatial dimensions of the output.



Padding

- Padding adds zeros around the border of the image.
- This helps preserve spatial information at the edges.
- Two common types:
  - Valid convolution: no padding
  - Same convolution: padding keeps output size equal to input

0	0	0	0	0	0	0	0
0	3	1	1	2	8	4	0
0	1	0	7	3	2	6	0
0	2	3	5	1	1	3	0
0	1	4	1	2	6	5	0
0	3	2	1	3	7	2	0
0	9	2	6	2	5	1	0
0	0	0	0	0	0	0	0

Output Size Formula

Output size of a convolution layer:

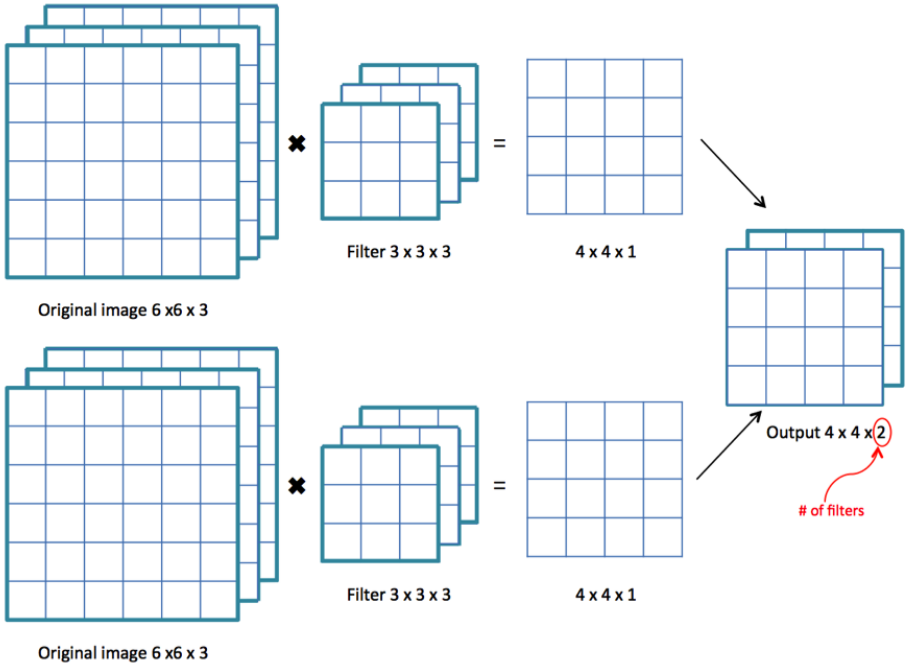
$$\left(\frac{n+2p-f}{s}+1\right) \times \left(\frac{n+2p-f}{s}+1\right)$$

- n = input size
- f = filter size
- p = padding
- s = stride

Usually filter sizes are odd numbers.

Multiple Filters

- CNNs use multiple filters simultaneously.
- Each filter detects different features.
- Outputs from filters are stacked to form an output volume.



Key idea

Each filter spans all input channels and produces one feature map. Using n'_c filters generates an output volume with n'_c channels.

Convolution with Multiple Filters

Input activation volume

$$n \times n \times n_c$$

Filter size

$$f \times f \times n_c$$

Output volume

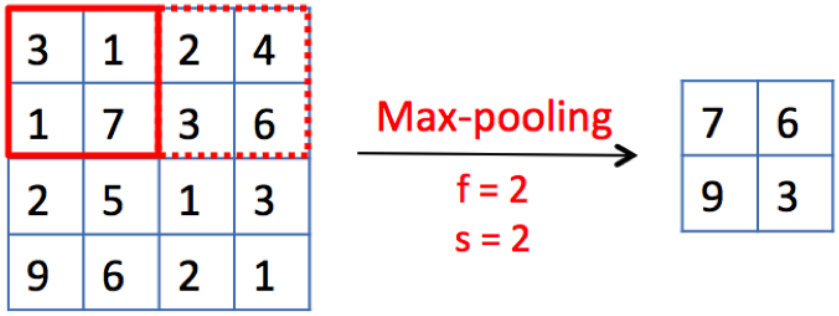
$$\left(\frac{n+2p-f}{s}+1\right) \times \left(\frac{n+2p-f}{s}+1\right) \times n'_c$$

- n : spatial size of the input
- n_c : number of input channels
- f : filter size
- s : stride
- p : padding
- n'_c : number of filters

Pooling Layers

Pooling layers reduce the spatial size of the feature maps.

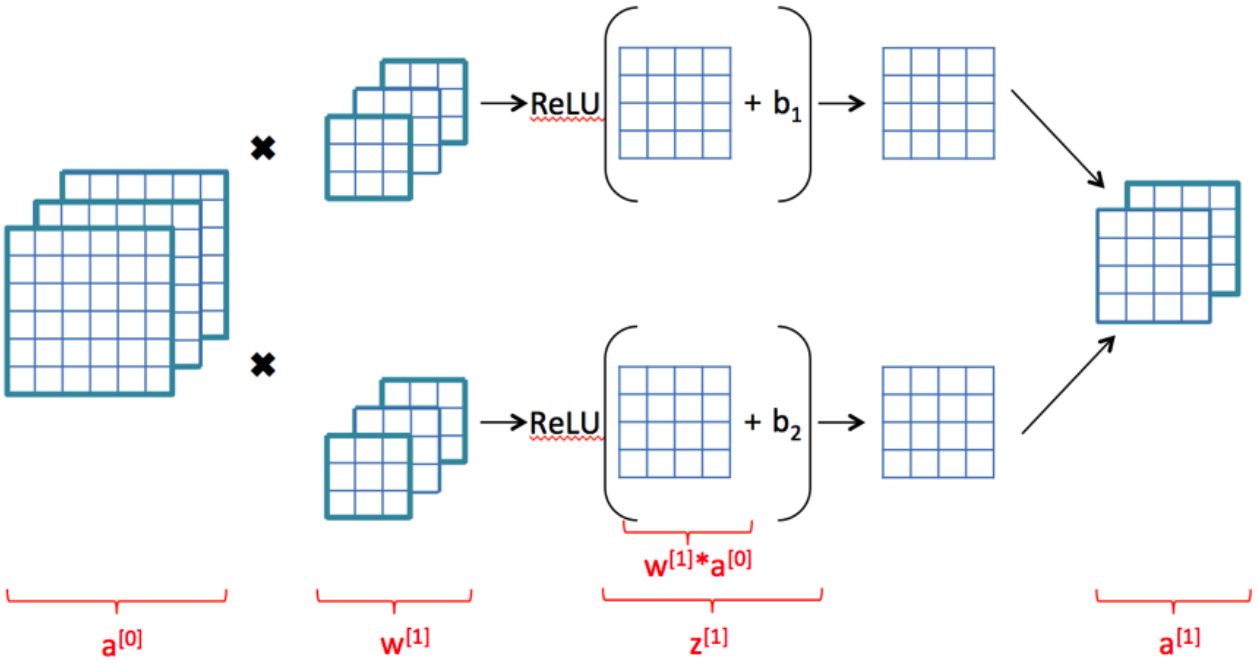
- Common types:
  - Max pooling
  - Average pooling
- Reduces the number of activations and computation.
- Helps keep the most important features.



Robustness to small spatial changes

Pooling makes the representation more stable to small shifts or distortions in the image. If a feature moves slightly, it can still produce a similar pooled response.

Activation Maps

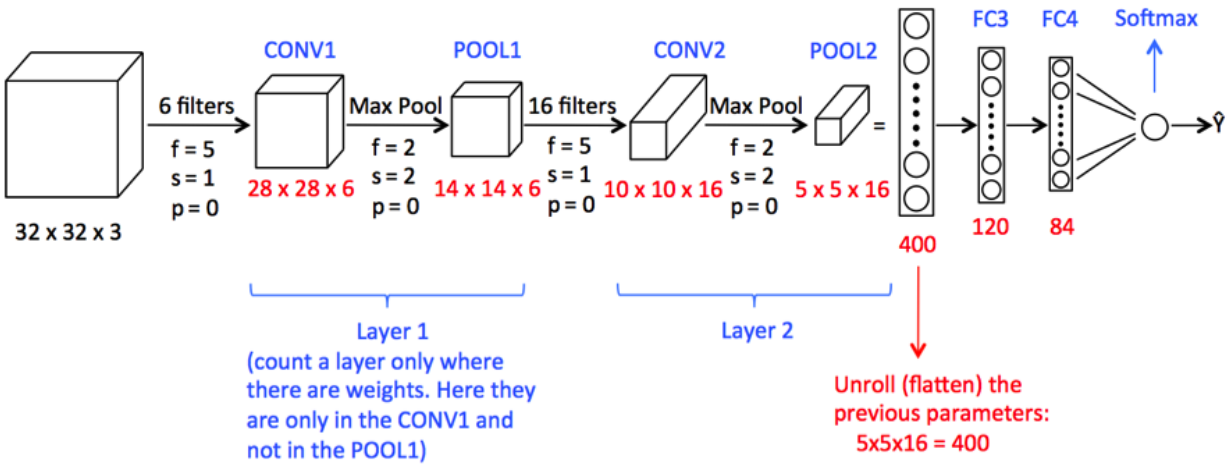


- Each filter slides over the input volume
- Computes a weighted sum + bias
- Applies ReLU activation
- Produces one feature map

Key idea

Different filters learn different features (edges, textures, shapes)

Typical CNN Architecture



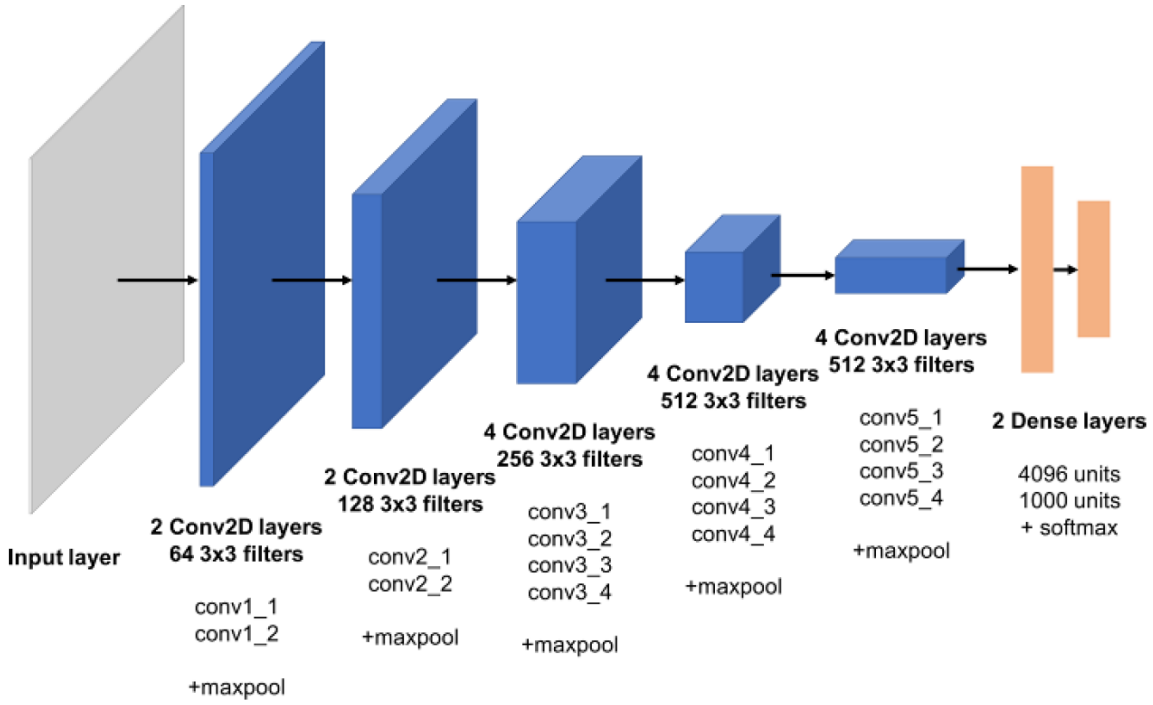
A common pattern:

- Convolution layer
- Pooling layer
- Convolution layer
- Pooling layer
- Fully connected layers
- Softmax classifier

Pretrained Models

Example: VGG19

- 19-layer convolutional neural network.
- Trained on more than 1 million images (ImageNet).
- Can classify images into 1000 categories.
- Widely used for **transfer learning**.



Conclusion

- CNNs are fundamental for computer vision.
- They efficiently process image data.
- Key layers: convolution, pooling and fully connected.
- Pretrained networks make deep learning more accessible.